Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Facade kalıbı (ekrandan, 00:16:30 ve 00:20:00)
  - Ad, Fransızca kökenli "facade" (bina cephesi) sözcüğünden gelir. Hoca bunu evlerin dış cephe kaplamasına benzetti: kalıbın anlamı bir arayüz, bir erişim noktası sağlamaktır.
  - Amaç: çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamak; böylece istekçiler alt sistemlerin alt düzey ayrıntılarından soyutlanır.
  - Kalıp yapısı: istekçi sınıflar (client classes) alt sistemdeki birçok sınıfın birçok metoduna tek tek erişmek yerine tek bir `Facade` sınıfıyla konuşur; `Facade` ilgili alt sistem sınıflarının metotlarını çağırır.
  - Bağlaşım hesabı: 3 istekçi ve 4 alt sistem sınıfı tam bağlı olursa $3 \times 4 = 12$ ilişki olur; araya Facade girince $3 + 4 = 7$ ilişkiye iner.
  - Kullanım: karmaşık bir alt sisteme basit bir arayüz sağlamak, alt sistemleri katmanlara ayırmak, bazı istekçilerin sistemin belli bir kısmıyla bağlaşımını azaltmak.
  - Basit bir kalıp olduğu için derste ayrıca kod yazılmadı.
- Flyweight kalıbı: amaç ve metin düzenleyici örneği (ekrandan, 00:30:00)
  - Ad, boksta en hafif sıklet kategorisinden gelir.
  - Amaç: paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemek ve bunu sistem kaynaklarını verimli kullanarak yapmak.
  - Dörtlü Çete örneği: her harfin en iyi görünüm için ayrı bir grafik nesnesi olarak gösterildiği bir metin düzenleyici. Her harf ayrı nesne olursa küçük bir belgede bile binlerce nesne olur; hocanın örneğinde 3 sayfalık bir belgede boşluklarla yaklaşık 5000 harf vardı, harf başına 1 KB'lık bitmap ile tek bir metin dosyası yaklaşık 5 MB tutar.
  - Çözüm: alfabedeki her harf (büyük ve küçük harf ayrı) tek bir nesne olarak ortak bir havuzda tutulur; metindeki her konum bu nesneye bir işaretçi saklar. Paylaşılan bu nesnelere flyweight denir.
  - Tasarım: soyut `Glyph` (`Draw(Context)`, `Intersects(Point, Context)`); alt sınıflar `Row`, `Column` (her ikisi de `children` tutar) ve `Character` (`char c`). Yapı Composite'i hatırlatır ama farklı bir kalıptır.
- Flyweight kalıp yapısı (ekrandan, 00:32:10)
  - `FlyweightFactory`, `flyweights` havuzunu tutar ve `GetFlyweight(key)` sunar: anahtara karşılık gelen flyweight varsa onu döndürür, yoksa yenisini oluşturur, havuza ekler ve döndürür.
  - Soyut `Flyweight`, `Operation(extrinsicState)` metodunu tanımlar. `ConcreteFlyweight` paylaşılır ve `intrinsicState` tutar; paylaşılması istenmeyen nesneler için kardeş sınıf `UnsharedConcreteFlyweight` (`allState`) kullanılabilir. `Client`, fabrikayla ve somut flyweight'lerle ilişkilidir.
  - Fabrika aynı anahtar için hep aynı nesneyi döndürdüğünden ayrıca Singleton kurmaya gerek yoktur. İstenirse somut sınıf fabrikanın private iç sınıfı yapılıp erişim daha da kısıtlanabilir (benzer bir yol Memento kalıbında da görülecek).
- İçsel ve dışsal durum bilgisi
  - Intrinsic state: nesneye özgü, paylaşılan, değişmeyen kısım (harfin resmi). Extrinsic state: dışarıdan, işlem sırasında parametre olarak verilen kısım (harfin çizileceği satır ve sütun).

  - Alternatif terimler: intrinsic yerine immutable, extrinsic yerine mutable. İçsel üyeler `final` tanımlanabilir.
  - Kazanç, içsel bilginin dışsal bilgiye oranı arttıkça artar.
  - Client, flyweight'i fabrikadan alır, dışsal durumu verir; flyweight başvurularını da client yönetir (metin örneğinde metin editörü).
- Flyweight'in kullanılabileceği anlar (ekrandan, 00:40:00)
  - Şu koşulların tümü doğru olmalıdır: uygulama çok fazla sayıda nesne kullanır; nesnelerin çokluğu nedeniyle saklama masrafı yüksektir; nesnelerin durum bilgisinin çoğu dışsallaştırılabilir; dışsal durumları ayrılan nesnelerin büyük bir grubunun yerini paylaşılabilecek az sayıda nesne alabilir; uygulama nesne kimliğine bağımlı değildir.
- Flyweight örneği: kimya fabrikası üretim bilgi sistemi (ekrandan, 00:50:00, 00:50:20, 00:53:40)
  - Esin kaynağı "Design Patterns Java Workbook". Fabrikada birçok kimyasal madde ve bunlardan oluşan çok sayıda karışım vardır; aynı madde birçok karışımda kullanılır. Bu yüzden Composite ve Decorator uygun değildir.
  - İlk çözüm: `KimyasalMadde` (`gram`, `isim`, `simge`, `atomAgirligi`; kurucu gram dışındaki üç bilgiyi alır; `getMol() = gram/atomAgirligi`) ve `Karisim` (`isim`, `LinkedList<KimyasalMadde> maddeler`, `maddeEkle`, `tarifEt`). Slayttaki örnek: 100 g barutta 75 g potasyum nitrat, 15 g karbon, 10 g kükürt.
  - Sorun: aynı madde farklı karışımlarda farklı gramajla yer alır; ya her karışım için maddenin tüm durum bilgisiyle yeni bir kopyası oluşturulur ya da tek nesne paylaşılırsa gram bilgisi en son eklenen karışımınkiyle ezilir.
  - İkinci çözüm ("daha doğru modelleme"): gram bilgisi ayrı bir `Bilesen` sınıfına alınır (`gram`, `madde`; `getGram()`, `getMol()`, `getMaddeAdi()`). `Karisim` artık `LinkedList<Bilesen>` tutar, `bilesenEkle(madde, gram)` ya da `bilesenEkle(Bilesen)` kullanır. Aynı madde bütün karışımlarda aynı nesne kimliğiyle yer alır; maddenin simgesi düzeltilirse tek yerde düzeltilir.
  - Flyweight çözümü: gram dışsal bilgi olarak öne çıkar ve madde nesnelerinin çok olacağı öngörülür. `MaddeFabrikasi`, `HashMap<String, KimyasalMadde> maddeler` ve `maddeBul(simge)` içerir (kolaylık için ikisi de static). `Karisim.bilesenEkle(simge, gram)` maddeyi fabrikadan simgeyle alır.
- Proxy kalıbı (ekrandan, 01:46:40 ve 01:50:00)
  - Amaç: bir nesneye erişimi denetlemek için onun yerine geçecek bir vekil nesneyi araya koymak.
  - Dörtlü Çete örneği: belge düzenleyicide büyük resimler yalnız ilgili sayfa gösterileceği zaman yüklenir ve çizilir. Soyut `Graphic` (`Draw()`, `GetExtent()`, `Store()`, `Load()`); alt sınıflar `Image` (`imageImp`, `extent`) ve `ImageProxy` (`fileName`, `extent`, `image` başvurusu). `ImageProxy.Draw()`: `image == 0` ise `image = LoadImage(fileName)`, sonra `image->Draw()`. `GetExtent()`: resim yüklenmemişse kendi `extent` değerini, yüklenmişse `image->GetExtent()` sonucunu döndürür.
  - Kalıp bileşenleri: `RealSubject` (erişimi denetlenecek ya da oluşturulması uzun sürecek asıl nesne), `Proxy` (asıl nesnenin yerine geçen vekil), `Subject` (ikisi için ortak arayüz).
  - Kullanım: bir nesneye basit bir işaretçiden fazlasının gerektiği her durum; gereken yetenekler proxy sınıfına kodlanır; işaret edilen nesne tamamen farklı bir adres uzayında (başka bir bilgisayarda) olabilir, örneğin dağıtık hesaplama.
- Proxy örneği: izledikçe öde video akış sunucusu (ekrandan, 01:57:15 ve 02:00:45)
  - Elimizdeki `StreamServer` yalnız belirli bir programı belirli bir hedefe gönderir; abonelik, ödeme, yaş sınırı gibi denetimler yoktur ve kaynak koduna erişilemez.
  - `<<interface>> StreamSource` (`stream(programID, destination): boolean`) hem `StreamServer` hem `StreamProxy` tarafından gerçeklenir. `StreamProxy`, `original: StreamSource` üyesini kurucuda alır; `stream()` içinde `checkPayment(...)` true ise `original.stream(...)` sonucunu, değilse `false` döndürür. `StreamDestination` (`server`, `address`, `makePayment`, `requestProgram`) proxy üzerinden ister.
  - Bu tür vekile Protection Proxy denir; nesne başka bir bilgisayardaysa vekile stub da denir.
- Davranışsal kalıplara giriş: Chain of Responsibility (ekrandan, 02:10:00 ve 02:13:10)
  - Amaç: birbiriyle ilişkili bir nesne zinciri ya da hiyerarşisi zaten varken, bir isteği karşılayamayan (ya da daha iyi karşılayacak biri bulunan) nesnenin isteği ilgili noktaya iletmesini sağlamak. İlk alıcının yanıt verememesi metodu olmamasından değil, yetki

ya da bilgi eksikliğinden kaynaklanır.
  - Benzetme: mağazada iade için çalışan, mağaza müdürü, bölge müdürü sırası.
  - Dörtlü Çete örneği: bağlama duyarlı yardım. `aPrintButton` ve `anOKButton` bir `aPrintDialog` içindedir; her nesnenin `handler` başvurusu bir üst bileşene gider, en sonda `anApplication` vardır. Zincirin başı özel (specific), sonu genel (general) yardımdır.
  - Kalıp yapısı: `Client` → soyut `Handler` (`HandleRequest()`, `successor` başvurusu), `ConcreteHandler1`, `ConcreteHandler2`. Client yanıtlanacak mesajın kaynağıdır; Handler mesajı tanımlayan arayüzdür ve zincir bağlantısını (successor) soyut sınıf olarak gerçekleyebilir; ConcreteHandler mesajı yanıtlar ya da bağlı olduğu başkasına iletir.
  - Kullanım: isteği karşılayabilecek birden fazla nesne varken hangisinin karşılayacağı önceden bilinmiyorsa; zincir çalışma anında kurulup değiştirilebilir.
- Chain of Responsibility örneği: reklam yönetim yazılımı (ekrandan, 02:20:00, 02:30:00, 02:36:05, 02:40:00)
  - Kampanya 1: spor sitesinin 18-39 yaş erkek ziyaretçilerine tıraş bıçağı, 18-39 yaş kadın ziyaretçilerine fitness salonu, diğerlerine spor mağazası indirimi reklamı. Kampanya 2: haber sitesinin 18-29 yaş A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamı, aksi halde Kampanya 1 aynen. Kampanya 3: coğrafi bölge ve gelir düzeyine göre lokanta reklamı.
  - `User` (cinsiyet, gelir düzeyi, yaş; `yasDahilMi(alt, ust)`), `Cinsiyet` ve `GelirDuzeyi` enum'ları, `Banner`.
  - Soyut `Reklam`: `sonraki` reklam ve `Banner` kurucuda verilir; soyut `reklamUygunMu(User)` ve `reklamGoster(User)`; `public final void reklamKotar(User u)` uygunsa kendini gösterir, değilse `sonraki != null` ise isteği `sonraki.reklamKotar(u)` ile iletir.
  - Somut sınıflar: `TrasBicak`, `FitnessSalonu`, `IndirimSporMagaza`, `OtoDergi`. Zincir dizi ile kurulur; her kullanıcıya uyan `IndirimSporMagaza` en sona, `sonraki = null` ile konur.
  - Kullanıcılar (Erkek, C, 23), (Erkek, A, 53), (Kadın, B, 33) için her iki kampanyada da sonuç: tıraş bıçağı, indirim, fitness. `OtoDergi` reklamı hiçbir kullanıcıya gösterilmedi, çünkü o gruba giren kullanıcı yoktu.

Hocanın Özellikle Vurguladığı Kısımlar

- Facade iç karmaşıklığı düzeltmez
  - Yalnız karmaşıklığın dışa yansımasını azaltır; bu yüzden "aman aman" bir kalıp değildir, hızlı bir çözüm olabilir.
  - Asıl yararı refactoring sırasında görülür: alt sistemde yapılan değişiklik yalnız Facade'ı etkiler, Facade'ın public metot imzaları korunduğu sürece istekçiler etkilenmez. Geriye uyumluluk önemliyse imzaları bozan refactoring yapılmamalıdır.
- Flyweight nesne kimliğini korumaz
  - Metindeki bütün küçük "a" harfleri bellekteki aynı nesneyi gösterir. Nesne kimliğinin önemli olduğu problemler (bir havayolunun taşıdığı yolcuların farklı kişiler olup olmadığı, bir reklamın 50 farklı kişiye mi yoksa bir kişiye 50 kez mi gösterildiği) için bu kalıp kullanılmamalıdır.
  - Kalıbın koşullarının hepsi doğru olmalıdır; biri bile sağlanmıyorsa ya kalıp uygulanamaz ya da kazanç yetersizdir. Az sayıda hacimli nesne için Flyweight mantıklı değildir.
- Fabrika paylaşımı garanti eder
  - Flyweight'ler paylaşılmazsa kalıbın ruhuna ters davranılmış ve bellek tasarrufu kaybedilmiş olur; bu yüzden kalıp fabrika kullanılmasını özellikle önerir.
- Veri en yakın ilişkili olduğu sınıfta saklanmalıdır
  - Hocaya göre bu temel bir nesneye yönelim kavramıdır. Gram bilgisinin doğru yeri `Bilesen`'dir; `KimyasalMadde`'de tutulursa madde farklı karışımlarda farklı gramajla yer aldığında belirsizlik doğar.
  - Statik üye çözüm değildir: isim, simge ve atom ağırlığı static yapılırsa biraz tasarruf olur ama aynı nesnenin farklı örnekleri yine yalnız gram bilgisi farklı olarak çoğalır.
- Proxy ile Adapter farkı
  - İkisinde de istek başka bir nesneye iletilir, ama amaçlar farklıdır. Adapter yalnız uyumsuz iki metot imzasını uyuşturur; Proxy araya erişim denetimi gibi ek işler koyar. Hoca bu farkın unutulmamasını istedi.
  - Örnekte `StreamServer` `StreamSource` arayüzünü gerçeklemiyorsa ve kaynak koduna

erişilemiyorsa Proxy değil Adapter yapılır.
  - Proxy vekalet ettiği nesneden önce ya da sonra iş yapabilir, ama Decorator'daki zincirleme ilişki burada yoktur.
- Chain of Responsibility'de zinciri doğru kurmak
  - Kalıbın en zayıf noktasıdır: zincir yanlış kurulursa isteğin yanıtlanacağının garantisi kalmaz. Hoca bunu bürokraside bir kapıdan ötekine gönderilip başa dönmeye benzetti.
  - Zincirin sonunda her kullanıcıya hizmet verebilecek bir eleman olmalıdır. Sonraki halkanın yanıt verip vermeyeceğini özyinelemeli sorgulamak mümkündür ama işi çok karmaşıklaştırır; zincirin doğru kurulduğundan emin olmak daha kolaydır.
  - Zincirin kurulmasını sağlayan ilişki en üst soyut sınıfta (tercihen hiyerarşinin kökünde) tutulmalıdır; aksi halde üst bileşenin türüne göre switch-case ya da if-else zinciri gerekir.
- Chain of Responsibility ile Decorator karışmamalıdır
  - Kampanya 2'nin Kampanya 1'e bir ön adım eklemesi Decorator'ı düşündürebilir. Ama Decorator'da bir nesne birden fazla yeteneğe sahip olur ve hepsi birlikte çalışır; burada ise doğrusal ilişkili seçenekler arasından yalnız en uygunu çalışır.
- `Reklam` ile somut alt sınıflar arasında Template Method kalıbı kurulmuştur (ekrandan, 02:40:00)
  - Kitabın sırası izlendiği için bu kalıp henüz görülmedi; ileride görülecek.

Kısa Tekrar Notları

- Facade: alt sistemin birçok arayüzü için tek ve üst düzey bir arayüz; bağlaşımı azaltır, iç karmaşıklığı azaltmaz.
- Facade bağlaşım örneği: 3 istekçi × 4 sınıf = 12 ilişki; Facade ile 3 + 4 = 7.
- Flyweight: çok sayıda küçük nesneyi paylaşarak bellek tasarrufu; `FlyweightFactory.GetFlyweight(key)` varsa var olanı, yoksa yenisini havuza ekleyip döndürür.
- Intrinsic (immutable, içsel, paylaşılan) ve extrinsic (mutable, dışsal, parametreyle verilen) durum.
- Flyweight nesne kimliğini korumaz; koşulların hepsi sağlanmalıdır.
- Kimya örneği: `KimyasalMadde`'den gram çıkarılıp `Bilesen`'e alındı; `MaddeFabrikasi.maddeBul(simge)` maddeleri paylaştırır.
- Proxy: `Subject` arayüzü, `RealSubject` ve onun yerine geçen `Proxy`. Örnekler: resmi gerektiğinde yükleyen `ImageProxy` ve ödeme denetleyen `StreamProxy` (Protection Proxy).
- Proxy ek denetim koyar; Adapter yalnız imza uyuşturur.
- GoF'un yedi yapısal kalıbı bu derste tamamlandı: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
- Chain of Responsibility: `Handler` (`HandleRequest()`, `successor`), istek yanıtlanana ya da zincir bitene kadar iletilir.
- Reklam örneğinde zincirin sonunda herkese uyan `IndirimSporMagaza` vardır; `reklamKotar` `final`dır.

Detaylı Açıklamalar

Facade

Facade kalıbı, istekçi sınıfların bir alt sistemin içindeki çok sayıda sınıfın çok sayıda metoduna farklı sıralarda ve farklı zamanlarda erişmek zorunda kaldığı durumu ele alır. Bu durumda istekçi tarafındaki birçok sınıf ile alt sistemdeki birçok sınıf arasında karmaşık bir çoktan çoğa ilişki oluşur. Kalıp, çağrılması gereken metotları kendi yazacağımız bir Facade sınıfının çağırmasını, istekçilerin de yalnız Facade ile konuşmasını önerir.

Hoca, kalıbın yararını ileride işlenecek refactoring konusuyla bağladı. Refactoring, dışarıya verilen hizmetleri bozmadan iç yapıyı düzeltmektir. Bazen daha iyi bir tasarım, dışarıya verilen hizmetlerin imzasını değiştirmeyi gerektirir ve testler bozulur. Araya önceden bir Facade konmuşsa iç yapıdaki değişiklik yalnız Facade'ı etkiler; Facade'ın public metot imzaları korunduğu sürece istekçiler hiç etkilenmez.

Flyweight

Metin düzenleyici örneğinde harfler bitmap olarak tutulur. Her konumda ayrı bir harf nesnesi oluşturmak yerine, her harf bir kez oluşturulup ortak havuza konur ve her konumdan bu nesneye bir işaretçi tutulur. Hoca işaretçinin yaklaşık 32 bit olacağını, bunun her harf için bir bitmap saklamaktan çok daha az yer kapladığını söyledi.

> **Ders sonrası not:** Hoca Unicode'daki karakter sayısını 1000-2000 olarak tahmin etti ve emin olmadığını söyledi. Unicode'un güncel sürümleri büyük olasılıkla 140.000'den fazla karakter tanımlar; örnek için önemli olan, bir metindeki harf sayısının alfabedeki farklı harf sayısından çok daha büyük olmasıdır.

Kalıp yapısında Flyweight'in işlemi, harfin çizileceği konum gibi dışsal bilgiyi parametre olarak alır ve kendi içsel bilgisini (harfin resmi) kullanarak işi yapar:

```
FlyweightFactory                        Flyweight (soyut)
  GetFlyweight(key)  --flyweights-->     Operation(extrinsicState)
                                             ^
       if (flyweight[key] exists)            |
         return existing flyweight;     +---+-----------------------+
       else                             |                           |
         create new flyweight;    ConcreteFlyweight       UnsharedConcreteFlyweight
         add it to pool;            Operation(extrinsicState)  Operation(extrinsicState)
         return new flyweight;      intrinsicState              allState

Client --> FlyweightFactory, ConcreteFlyweight, UnsharedConcreteFlyweight
```

(ekrandan, 00:32:10)

Kimya Fabrikası Örneği

İlk çözümde gram bilgisi `KimyasalMadde` sınıfındadır. Derste ilk çözüm çalıştırıldığında gramlar 0 çıktı, çünkü örnek kodda `setGram` çağrısı unutulmuştu; hoca kodu iki biçimde düzeltti: `MainApp` içinde her madde için `setGram` çağırıp `maddeEkle` ile eklemek (çözüm 1a) ya da `maddeEkle(KimyasalMadde, double gram)` biçiminde ikinci parametre eklemek (çözüm 1b). Her iki durumda da gram bilgisi maddeye dışarıdan verilir.

Ardından hoca ilk çözümün sorununu canlı gösterdi: aynı karbon nesnesi hem kara barutta 15 g hem de ikinci bir barut karışımında 14 g olarak kullanıldığında, karbon nesnesinde yalnız en son eklendiği karışımdaki gram (14) kalır; önceki bilgi kaybolur (ekrandan, 02:00:00).

İkinci çözümün `Bilesen` sınıfı (ekrandan, 01:10:00):

```java
public class Bilesen {
    private double gram;
    private KimyasalMadde madde;

    public Bilesen( double gram, KimyasalMadde madde ) {
        this.gram = gram; this.madde = madde;
    }
    public double getGram( ) { return gram; }
    public double getMol( ) { return madde.getMol(gram); }
    public String getMaddeAdi( ) { return madde.getIsim( ); }
}
```

Çözüm 2 ile çözüm 3 karşılaştırıldığında `Bilesen` ve `KimyasalMadde` değişmez; yalnız `Karisim.bilesenEkle` maddenin kendisi yerine anahtarını (simgesini) alır ve maddeyi `MaddeFabrikasından` bulur (ekrandan, 01:20:00).

> **Ders sonrası not:** Slayttaki tarifte 100 g barutta 75 g potasyum nitrat vardır; derste çalıştırılan kodda ise "Kara barut" için sodyum nitrat, ikinci karışım ("Temiz barut") için potasyum nitrat kullanıldı. Klasik kara barut tarifinde büyük olasılıkla potasyum nitrat (KNO_3) kullanılır. Bu fark kalıbın anlatımını etkilemez.

Proxy

Hoca Proxy'yi eski kelime işlemcilerde ve yavaş internet döneminde resimlerin yerinde görülen, resmin boyutunda ve köşesinde kırmızı çarpı olan dikdörtgenlerle anlattı: program yanıt süresini kısa tutmak için resimleri hemen yüklemez, yalnız gösterilen sayfanın (ve belki birkaç sayfa öncesinin ve sonrasının) resimlerini yükler. Gerçek resim ile vekil resmin aynı biçimde kullanılabilmesi için ikisi de `Graphic` arayüzünü gerçekler.

Video akışı örneğinde `StreamProxy` şöyledir (ekrandan, 02:00:45):

```
public class StreamProxy implements StreamSource {
    private StreamSource original;

    public StreamProxy(StreamSource original) {
        this.original = original;
    }

    public boolean stream(String programID, String destination) {
        if( checkPayment(programID, destination) )
            return original.stream(programID, destination);
        else
            return false;
    }
    ...
}
```

Hoca örnek çözümde proxy'nin somut `StreamServer` yerine ortak arayüz `StreamSource` ile ilişkilendirildiğini, böylece farklı türde sunuculara da vekalet edebildiğini söyledi; Dörtlü Çete'nin yapısında ise proxy doğrudan gerçek nesneye bağlıdır. İkisi de olabilir.

Chain of Responsibility

Eski ofis programlarında yardım düğmesine basıldıktan sonra fare imleci soru işaretli hale gelir, tıklanan alanla ilgili odaklanmış yardım gösterilirdi. Tıklanan bileşenin yardımı kodlanmamışsa bir üst bileşenin yardımı gösterilir; iletim, en iyi yardımı verebilecek yer bulunana ya da zincirin sonuna gelinene kadar sürer. Bunun için her bileşenin hangi üst bileşenin içinde olduğunu bilmesi gerekir; böyle doğal bir ilişki yoksa ilişki zinciri ayrıca kurulmalıdır.

Reklam örneğindeki soyut sınıfın zinciri yöneten metodu (ekrandan, 02:30:00):

```
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
            + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
            + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}
```

Kampanya 2'nin zinciri (ekrandan, 02:40:00):

```
public void kampanya2( ) {
    Reklam[] reklamlar = new Reklam[4];
    reklamlar[3] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[2] = new FitnessSalonu(new Banner(), reklamlar[3]);
    reklamlar[1] = new TrasBicak(new Banner(), reklamlar[2]);
    reklamlar[0] = new OtoDergi(new Banner(), reklamlar[1]);
```

```
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
```

Hoca Kampanya 2'yi Kampanya 1'in dizisine bir eleman ekleyerek kurmak yerine bağımsız tuttu; bunun biraz kod tekrarı yarattığını kabul etti.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.